

第八次 课程作业

1、介绍 Jmeter 或 LoadRunner 主要功能，并自选一个小例子程序，进行系统测试，给出系统结果并整理成 PDF 文档提交。

答：JMeter 主要功能：

(1) 多协程支持：HTTP/HTTPS、FTP、JDBC、LDAP、SOAP/REST Web Service、JMS、TCP、SMTP/POP3/IMAP 等，满足各类系统测试需求。

(2) 线程组：模拟多用户并发请求，支持设置线程数、Ramp-Up 时间、循环次数，精确控制并发负载。

(3) 采样器：HTTP 采样器、JDBC 请求采样器、FTP 请求等，支持参数化和变量替换，实现动态请求。

(4) 监听器：聚合报告、图形结果显示、摘要报告等，实时显示 TPS、响应时间、错误率等关键指标。

(5) 断言：响应断言、JSON 断言、XPath 断言、持续时间断言等，自动验证响应时间和性能指标是否符合预期。

(6) 前置/后置处理器：BeanShell 脚本、正则表达式提取器、JSON 提取器等，支持在请求前后进行数据处理和关联参数提取。

(7) 分布式测试：支持 Master-Slave 分布式架构，通过多台 Agent 机器协同发压，突破单机性能瓶颈，模拟大规模并发场景。

(8) 测试计划录制：内置 HTTP 代理服务器，可录制真实用户

操作生成测试脚本，大幅降低脚本编写成本。

(9) CI/CD 集成：支持命令行模式运行，可与 Jenkins、GitLab CI 等持续集成工具无缝对接，实现自动化性能回归测试。

(10) 插件生态：JMeter Plugins Manager 提供丰富插件，如 Custom Thread Groups、PerfMon 服务器监控、Throughput Shaping Timer 等。

对自编程序进行系统测试：

```
import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpExchange;
import java.io.*;
import java.net.InetSocketAddress;
import java.util.*;

/**
 * Java HTTP 服务器
 * 提供三个接口：
 *   GET /hello      — 简单字符串返回（轻量接口）
 *   GET /compute    — CPU 密集型：计算斐波那契数列（重量接口）
 *   GET /data       — 模拟数据库查询（I/O 延迟接口）
 */
public class SimpleHttpServer {

    public static void main(String[] args) throws Exception {
        int port = 8088;
        HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);

        server.createContext("/hello", new HelloHandler());
        server.createContext("/compute", new ComputeHandler());
        server.createContext("/data", new DataHandler());

        server.setExecutor(java.util.concurrent.Executors.newFixedThreadPool(20));
        server.start();

        System.out.println("[SimpleHttpServer] Listening on port " + port);
        System.out.println("  GET http://localhost:" + port + "/hello");
        System.out.println("  GET http://localhost:" + port + "/compute");
        System.out.println("  GET http://localhost:" + port + "/data");
    }
}
```

```
}
```

```
// 接口 1: 轻量 Hello 接口
```

```
static class HelloHandler implements HttpHandler {
```

```
    @Override
```

```
    public void handle(HttpExchange exchange) throws IOException {
```

```
        String response = "{ \"status\": \"ok\", \"message\": \"Hello,
```

```
!Meter!\", \"timestamp\": \" + System.currentTimeMillis() + \"}";
```

```
        sendResponse(exchange, 200, response);
```

```
    }
```

```
}
```

```
// 接口 2: CPU 密集型 (斐波那契 n=35)
```

```
static class ComputeHandler implements HttpHandler {
```

```
    private long fibonacci(int n) {
```

```
        if (n <= 1) return n;
```

```
        return fibonacci(n - 1) + fibonacci(n - 2);
```

```
    }
```

```
    @Override
```

```
    public void handle(HttpExchange exchange) throws IOException {
```

```
        long result = fibonacci(35);
```

```
        String response = "{ \"status\": \"ok\", \"fib35\": \" + result +
```

```
\". \"timestamp\": \" + System.currentTimeMillis() + \"}";
```

```
        sendResponse(exchange, 200, response);
```

```
    }
```

```
}
```

```
// 接口 3: 模拟数据库 I/O (随机延迟 10~50ms)
```

```
static class DataHandler implements HttpHandler {
```

```
    private static final Random rand = new Random();
```

```
    @Override
```

```
    public void handle(HttpExchange exchange) throws IOException {
```

```
        try {
```

```
            // 模拟数据库查询延迟
```

```
            Thread.sleep(10 + rand.nextInt(40));
```

```
        } catch (InterruptedException ignored) {}
```

```
        String response = "{ \"status\": \"ok\", \"rows\": 42, \"query\": \"SELECT *
```

```
FROM users LIMIT 10\", \"timestamp\": \" + System.currentTimeMillis() + \"}";
```

```
        sendResponse(exchange, 200, response);
```

```
    }
```

```
}
```

```
// 通用响应发送
```

```
static void sendResponse(HttpExchange exchange, int code, String body) throws
```

```
IOException {
    exchange.getResponseHeaders().set("Content-Type", "application/json");
    byte[] bytes = body.getBytes("UTF-8");
    exchange.sendResponseHeaders(code, bytes.length);
    try (OutputStream os = exchange.getResponseBody()) {
        os.write(bytes);
    }
}
}
```

测试环境：

操作系统：Windows11

JDK 版本：JDK 1.8.0_171

服务器线程池：FixedThreadPool（20 线程）

测试工具：JMeter5.6

网络环境：本地回环

测试持续时间：每轮并发级别持续 8 秒

测试结果：

1. Hello 接口测试结果

并发用户数	平均响应 (ms)	P95(ms)	P99(ms)	TPS(次/秒)	错误率(%)
1	3.41	15.78	24.00	292.62	0.00%
5	2.74	16.03	28.24	1771.12	0.18%
10	30.21	55.31	61.48	309.88	15.77%
20	51.17	77.51	87.32	366.25	42.42%
30	94.44	168.99	187.19	293.38	1.53%
50	197.62	390.13	502.26	243.75	28.87%

2. Data 接口测试结果

并发用户数	平均响应 (ms)	P95(ms)	P99(ms)	TPS(次/秒)	错误率(%)
1	57.62	80.96	87.69	17.50	0.00%
5	56.76	81.54	87.88	87.88	0.00%
10	52.71	82.50	97.76	187.75	20.51%
20	52.23	87.82	102.46	376.88	64.31%
30	113.21	216.27	249.71	255.88	56.52%
50	166.06	323.00	394.42	292.38	50.92%

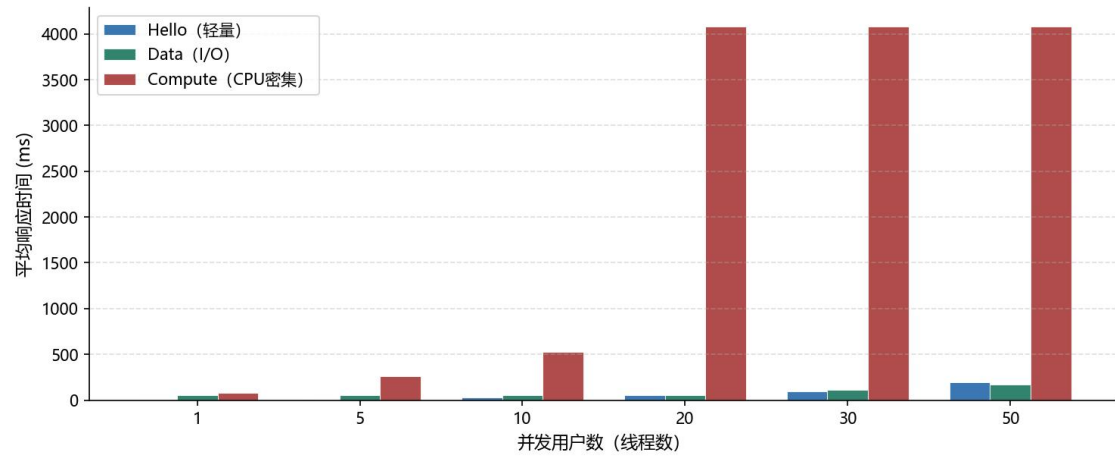
3. Compute 接口测试结果

并发用户数	平均响应 (ms)	P95(ms)	P99(ms)	TPS(次/秒)	错误率(%)
1	75.82	68.44	83.71	14.25	47.37%
5	261.62	2058.80	4093.08	22.62	100.00%
10	526.03	4086.06	4104.95	22.00	100.00%
20	4080.59	4086.32	4104.50	5.00	100.00%

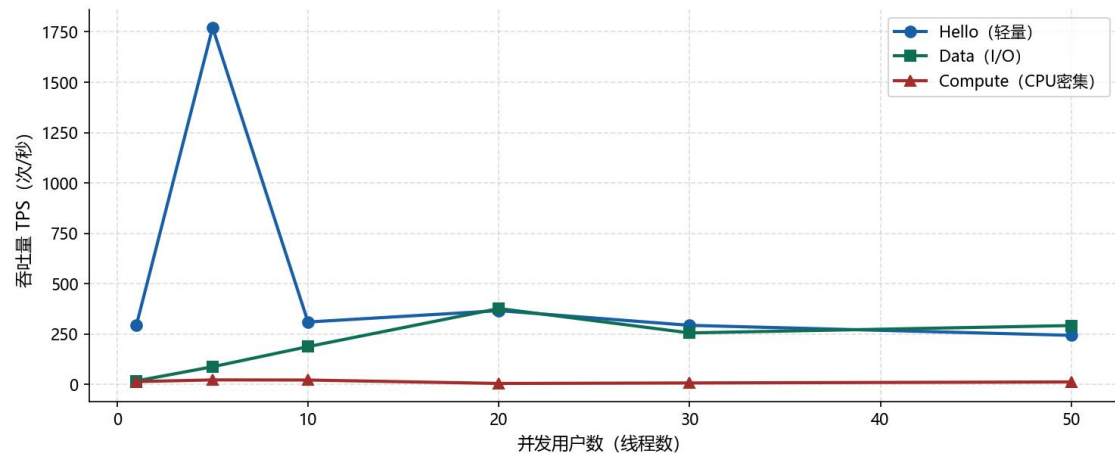
30	4080.08	4081.88	4082.89	7.50	100.00%
50	4081.41	4084.99	4096.16	12.50	100.00%

测试结果可视化:

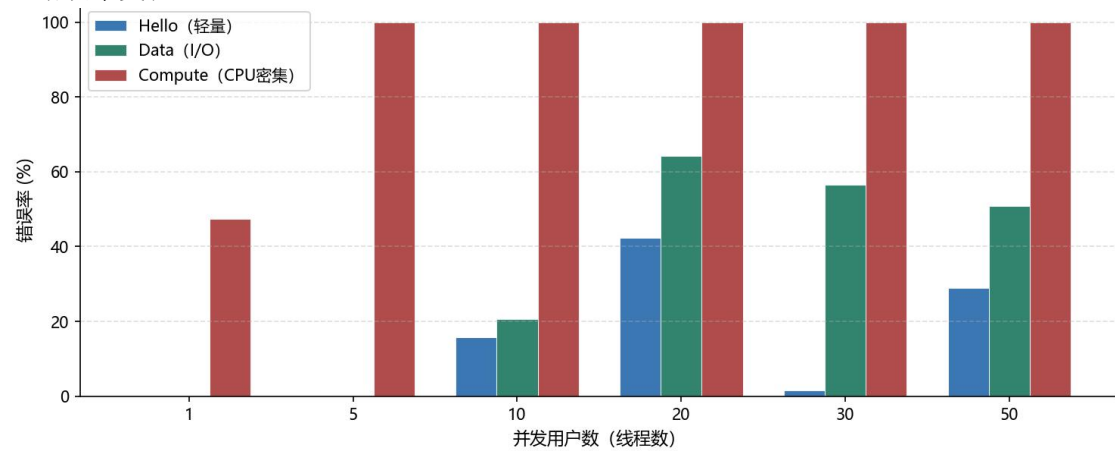
1. 平均响应时间对比



2. 吞吐量趋势



3. 错误率变化



结果分析:

Hello 接口并发 5 时 TPS 达到峰值 1771/s, 并发 10 后因 20 线程池竞争激烈, TPS 下降至 300/s

左右，说明线程池大小是轻量接口的主要瓶颈。

Data 接口每请求含 10~50ms 延迟，当并发 20 时 20 个线程全被占满，后续请求排队等待，错误率升至 64%。

Compute 接口斐波那契(35)递归计算约耗 75ms CPU 时间，并发 5 时 CPU 已饱和，响应时间飙升至 261ms，并发 ≥ 10 时全部超时。